



СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

Основы технического зрения Операции с изображениями и видео

1.1 ОСНОВЫ ТЕХНИЧЕСКОГО ЗРЕНИЯ

Модуль 1.1 посвящен базовым вопросам технического зрения. В первой части модуля будут рассмотрены общие вопросы технического зрения: в чем суть данной дисциплины, какие задачи решает техническое зрение и с какими исходными данными работает. Вторая часть модуля посвящена базовым практическим вопросам: какой инструментарий можно использовать при решении задач технического зрения, как представляются изображения в памяти и как выполнять такие операции над изображениями, как чтение, отображение на экране и сохранение на диск.

1.1.1 Введение в техническое зрение

Чтобы понять, что такое техническое зрение и какие задачи ставятся перед системами технического зрения, можно начать с ответа на вопрос: какую информацию получает человек, когда видит какое-то изображение? Например, посмотрим на рис. 1.1.



Рисунок 1.1 – Катер

Какую информацию может извлечь из этого изображения человек? Нам понятно, что изображение – это фотография катера. Также очевидно, что нижняя часть изображения – это какой-то водоем. Во-первых, мы видим рябь на поверхности. Значит, это какая-то жидкость. Во-вторых, по ней плывет катер. После этого утверждения возникает следующий вопрос: откуда мы знаем, что катер движется? Он оставляет за собой характерный след. Также мы можем извлечь и

другую контекстную информацию из изображения. Например, мы знаем примерный размер катера и можем сопоставить вид катера и вид берега. Можно примерно предположить, на каком расстоянии от берега находится катер. Таким образом, мы получили контекстную информацию о размерах объектов на изображении и расстоянии. Также на изображении можно заметить полосу суши с лесом. Значит, в верхней части изображения мы видим небо. Получается, мы также можем разметить изображение.

Человек может извлечь множество информации из изображений, так как помимо зрения использует другие способы обработки информации: ассоциативные связи, опыт, способности к распознаванию, логическое мышление и так далее. Однако с точки зрения компьютера любое изображение – это только набор точек определенного цвета. Изображение в памяти будет представлено матрицей, где каждый элемент будет описывать цвет точки, а координаты элемента – ее положение. Максимум информации, которую можно извлечь без дополнительной программной обработки – размер изображения и число каналов, которыми описывается цвет.

Очевидно, что разница в восприятии изображения человеком и компьютером огромна. Техническое зрение же занимается сведением этой разницы к минимуму.

Техническое зрение – это дисциплина, которая является частью искусственного интеллекта. Задача технического зрения – извлечение информации из изображений (или видеопотока, который тоже состоит из изображений). И у этой задачи есть множество ответвлений. Можно заниматься обработкой изображений с целью удаления шума, повышения их качества, подготовки к следующим этапам. Можно заниматься выделением знаковых частей изображения – например, границ, каких-либо геометрических фигур, определенных областей. Можно восстанавливать трехмерные сцены по нескольким изображениям, вычислять размеры объектов или расстояние до них. Так как техническое зрение решает множество задач, оно находит применение в самых разных областях – от робототехники до астрономии.

На первом этапе знакомства с техническим зрением необходимо понять, что такое изображение и как оно представляется с точки зрения данных. На рис. 1.2 представлено некоторое цветное изображение. Каждый квадрат на изображении символизирует один пиксель.



Изображение в памяти хранится в виде матрицы. Левый верхний пиксель изображения I будет иметь координаты $I(0, 0)$. Его сосед справа будет иметь координаты $I(1, 0)$, сосед снизу – $I(0, 1)$. Координату x пикселя описывает номер столбца той матрицы, где хранится изображение. И, соответственно, координата y описывается номером строки той же матрицы. Цвет пикселя кодируется некоторым числом или набором чисел. Попробуем представить, как выглядит матрица нашего изображения. Для этого на рис. 1.3 отобразим два варианта матрицы. На рис. 1.3, *а* проиллюстрируем промежуточный этап: каждый пиксель опишем числом, сохранив для наглядности цвет. А на рис. 1.3, *б* покажем, как по-настоящему выглядит изображение в памяти.



6

Рисунок 1.3 – Изображение в памяти:

а – данные, дополненные цветом; б – настоящие данные

Как хранится изображение в памяти – ясно. Теперь нужно понять, как именно числа описывают цвет пикселей. Известно, что при рисовании красками используют три цвета для получения остальных цветов: красный, синий и желтый. Результат определяется тем, в каких пропорциях смешивались основные цвета. Когда мы видим рисунок на бумаге, мы воспринимаем цвет за счет света, который отражается от листа.

Цвет пикселей на изображении, которое мы видим на экране монитора, формируется иным способом. Если каждая точка на бумаге отражает свет, то каждый пиксель монитора свет излучает. Из-за этого отличия – излучение света вместо отражения – цветовая модель на основе красного, синего и желтого цветов не подходит для формирования нужного цвета на экране.

Наиболее распространенная цветовая модель, используемая для вывода изображений на экраны – **RGB**. Она включает три канала: красный – **R**, зеленый – **G** и синий – **B**. Каждый канал описывает одно число – его **яркость**. Эта величина относительная. Минимальное значение яркости канала – 0 – означает, что канал не излучает свет вообще. Максимальное значение яркости – N – указывает, что излучение света предельно возможное. Весь доступный диапазон описывается числами с шагом 1. Чем больше значение N , тем больше бит нужно, чтобы описать яркость канала. В цветовых моделях на каждый канал отводится одинаковое количество бит. Например, в RGB на каждый канал отводится 8 бит. Значит, каждый пиксель описывает три восьмибитных числа – 24 бита. То, сколько бит отводится на описание цвета пикселя, называют **глубиной цвета**.

Помимо RGB, существуют другие цветовые модели – например, CMYK или HSB. На них подробно останавливаться не будем.

Если в RGB каналы имеют одинаковую яркость, получается серый цвет. Чем меньше значение яркости, тем он темнее, чем больше – тем светлее. Крайние значения – белый (все каналы имеют максимальную яркость) и черный (все каналы имеют яркость 0). Иногда такое представление бывает полезно, когда информация о цвете не важна. В таком случае хранить три одинаковых числа для описания **оттенка серого** бессмысленно. Для описания таких изображений существует серая шкала – **grayscale**. Минимальное значение яркости в этой шкале – 0, максимальное – 255. Следовательно, глубина цвета – 8. Такие изображения занимают в три раза меньше памяти, чем цветные в RGB, и часто используются в техническом зрении.

1.1.2 Введение в OpenCV

Задачи технического зрения разнообразны, и для их решения требуется хорошая теоретическая подготовка и навыки программирования. Одного понимания того, как работает алгоритм, недостаточно, чтобы реализовать его на практике: важно учитывать и объем данных, и скорость их обработки, и множество других неочевидных нюансов. В преодолении этих проблем частично может помочь использование специальных библиотек компьютерного зрения. При этом необходимо, чтобы лицензия, по которой распространяется библиотека, разрешала использование в тех целях, для которых создается программный продукт.

OpenCV (*Open Source Computer Vision*) – библиотека компьютерного зрения с открытым исходным кодом, тем не менее, допускает использование в коммерческих продуктах. Она предоставляет базовые средства для решения задач технического зрения. Это и помогает в решении сложных частей задачи, и одновременно дает простор для дополнения получившихся решений разработчиком.

OpenCV реализована на C/C++, и исходный код можно посмотреть на официальном репозитории: <https://github.com/opencv/opencv>. До версии 3.0 библиотека была монолитной, однако затем была разбита на две части: **opencv** и **opencv_contrib**. Исходный код второй части хранится в отдельном репозитории: https://github.com/opencv/opencv_contrib. В него вынесены модули с передовой функциональностью: распознавание лиц, текста, биокomпьютерное зрение, глубокие нейронные сети и другие.

Достоинством OpenCV является подробная документация. Ссылка на документацию, форум по ответам на вопросы и домашнюю страницу доступны в Readme репозитория.

При работе с OpenCV – сборке или установке, чтении документации, использовании средств библиотеки и т. д. – необходимо учитывать то, какая версия библиотеки используется. В некоторых моментах отличия могут быть существенными. Библиотеку можно как собрать из исходников, так и установить из бинарников. Процесс сборки / установки зависит от системы и версии библиотеки. Он подробно описан в документации, поэтому здесь эти вопросы рассмотрены не будут. Использовать OpenCV можно не только при программировании на C/C++. Мы будем рассматривать использование **OpenCV 4.2.0** при программировании на **Python 3** (далее Python).

OpenCV в скрипте на Python подключается так же, как любая другая библиотека: через **import**. Однако есть один момент, который нужно учитывать. OpenCV 3 действительно подключается просто:

```
import cv2
```

Здесь число 2 в **cv2** указывает **НЕ** на версию библиотеки. Это связано с наименованием каталогов библиотеки. Подключение OpenCV 4 будет отличаться:

```
import cv2.cv2
```

Такое отличие связано с тем, что расположение файлов в каталогах в версии 4 поменялось. Очевидно, для удобства можно использовать такой способ подключения:

```
import cv2.cv2 as cv2
```

Для проверки того, что библиотека установилась корректно для правильной версии Python, можно написать небольшой скрипт, который выводит версию библиотеки:

```
import cv2.cv2 as cv2  
print(cv2.__version__)
```

Первое, с чего стоит начать изучение технического зрения – базовые навыки работы с изображениями. Чтение изображений выполняется с помощью функции **imread()**.

```
img = cv2.imread(  
    path,                                # путь до изображения  
    flags=cv2.IMREAD_COLOR              # параметр(ы) чтения  
)
```

Первый аргумент функции – **path** – путь до изображения. Его нужно задавать таким образом, чтобы изображение было доступно из того каталога, откуда запускается файл скрипта. Самый удобный способ – писать полный путь до изображения. Путь задается в виде строки. При этом нужно помнить, что разделитель в пути в Windows – символ «\» – необходимо экранировать.

Второй аргумент – **flag** – определяет, как будет прочитано изображение. Все возможные значения этого аргумента перечислены в документации. По умолчанию аргументу присвоен флаг **cv2.IMREAD_COLOR** – конвертация в трехканальное цветное изображение в цветовой модели **BGR**. Цветовая BGR

совпадает с RGB во всём, кроме порядка цветов. В BGR первый канал синий, второй – зеленый, третий – красный. На каждый канал отводится 8 бит. Также можно прочитать изображение в оттенках серого – `cv2.IMREAD_GRAYSCALE`, уменьшить изображение при чтении – например, `cv2.IMREAD_REDUCED_COLOR_2` и аналоги для уменьшения в 4 и 8 раз, а также для уменьшения при чтении в оттенках серого.

Возвращаемое значение функции – изображение. В OpenCV на C/C++ изображения описывает класс **Mat**. Тем не менее, если вывести тип возвращаемого значения – переменной **img**, мы увидим: `class 'numpy.ndarray'`. В случае если изображение не было прочитано, в переменной будет храниться значение **None**.

Numpy – библиотека с открытым исходным кодом для Python. Она поддерживает работу с многомерными массивами и позволяет выполнять над ними высокоуровневые математические операции. И, соответственно, **ndarray** – многомерный массив (*N-dimensional array*). В зависимости от того, сколько каналов у изображения, массив **img** будет двумерным (1 канал) или трехмерным (3 канала).

Еще один полезный метод – сохранение изображения. Для того чтобы сохранить изображение на диск, можно воспользоваться функцией **imwrite()**.

```
res = cv2.imwrite(  
    filename,      # путь до файла сохранения  
    img,           # переменная, хранящая изображение  
    flags          # параметр(ы) сохранения  
)
```

Первый аргумент функции – **filename** – путь до файла, куда нужно сохранить изображение. Если такого файла не существует, он будет создан. Если указан существующий файл, он будет перезаписан. Второй аргумент – **img** – переменная изображения, которое сохраняется. Последний аргумент – **flags** – перечень флагов.

Обычно эта функция позволяет сохранять одноканальные или трехканальные RGB изображения, в которых на каждый канал отводится 8 бит. Другие возможные варианты сохранения подробно описаны в документации. Возвращаемое значение – булево, *True* при успешном сохранении и *False* при неуспешном.

Помимо чтения и сохранения изображений, полезно уметь отображать их в окне во время исполнения программы. Создать окно можно с помощью функции **namedWindow()**.

```
cv2.namedWindow(  
    name,                                # имя окна  
    flags=cv2.WINDOW_AUTOSIZE          # параметр(ы) окна  
)
```

В первый аргумент – **name** – передается строка с именем окна. Это имя будет его идентификатором в других операциях. Во второй аргумент – **flags** – передаются параметры создаваемого окна. По умолчанию задан параметр `cv2.WINDOW_AUTOSIZE` – подбирать размер окна под размер изображения. Полный список параметров описан в документации.

Две парные функции – удаление одного или нескольких окон: **destroyWindow()** и **destroyAllWindows()**, соответственно. Функция **destroyWindow()** принимает один аргумент – строку, содержащую имя окна. Функция **destroyAllWindows()** не требует аргументов.

Для отображения изображения в определенном окне можно воспользоваться функцией **cv2.imshow()**.

```
cv2.imshow(  
    winName, # имя окна  
    img      # переменная, содержащая изображение  
)
```

Первый аргумент – **winName** – принимает строку, содержащую имя окна. Если такое окно уже было создано функцией **namedWindow()**, изображение будет в этом окне. Если такого окна еще нет, оно будет создано автоматически и в качестве идентификатора будет иметь строку – имя. Такой способ, очевидно, не позволяет настроить параметры окна, поэтому подходит не для всех случаев. Второй аргумент – **img** – переменная, содержащая изображение. В зависимости от размера изображения оно может быть отмасштабировано. Поведение функции в зависимости от глубины подробно описано в документации.

Хоть функция **cv2.imshow()** и предназначена для отображения изображений, ее недостаточно для того, чтобы вывести изображение на экран. Функция – одна или несколько – обязательно должны сопровождаться вызовом функции **waitKey()**. Данная функция предназначена для того, чтобы отображать изображение в течение нескольких миллисекунд.

```
key = cv2.waitKey(  
    time      # время ожидания нажатия  
)
```

Единственный аргумент функции – **time** – время ожидания в миллисекундах. Если необходимо, чтобы функция отображалась сколь угодно долго до нажатия на клавишу, можно передать в аргумент значение меньше 1 (например, 0). Возвращаемое значение – **key** – код клавиши, которая была нажата.

1.2 ОПЕРАЦИИ НАД ИЗОБРАЖЕНИЯМИ И ВИДЕОПОТОКОМ

В данном модуле рассматриваются дополнительные возможности работы с изображениями, которые могут быть полезны на этапе создания и отладки приложений. Первая часть модуля посвящена особенностям работы с изображениями в OpenCV при программировании на Python. Изображения в OpenCV на C/C++ представляются как экземпляры класса `Mat`. Однако при программировании на Python изображение будет многомерным массивом, описанным в библиотеке NumPy. Данная библиотека предназначена для выполнения высокоуровневых операций над матрицами, которые можно с пользой применить при работе с изображениями. Вторая часть модуля освещает вопросы изменения изображений уже средствами OpenCV: изменению цветовой модели, рисованию простых геометрических фигур, добавлению текста, работе с отдельными каналами. Также во второй части рассматривается работа с видеопотоком как с последовательностью изображений.

1.2.1 Использование NumPy при работе с изображениями

Для чтения, отображения и сохранения подключать библиотеку NumPy не нужно. Это стоит делать тогда, когда планируется использовать в программе возможности работы с изображениями, которые применимы к любым многомерным массивам. Данная библиотека также требует предварительной установки. На официальном сайте <https://numpy.org/> можно найти описание библиотеки, документацию к ней, а также руководство по использованию и инструкцию по установке.

Подключение библиотеки выполняется так же, как и подключение любого другого модуля:

```
import numpy
```

Для удобства можно сократить:

```
import numpy as np
```

Одна из наиболее полезных операций над изображениями – копирование изображения. Часть функций по работе с изображениями, особенно связанные с рисованием, изменяют входное изображение. Кроме того, при отладке этапов, связанных с предварительной обработкой, полезно сравнить результат обработки с исходным изображением. Чтобы иметь несколько вариантов одного и того же изображения, не обязательно считывать его несколько раз. Для этого можно воспользоваться функцией `copy()` из библиотеки NumPy.

```
img_copy = np.copy (
    img,          # входное изображение
    order='K'     # способ копирования
)
```

Первый аргумент функции – **img** – изображение, копию которого нужно создать. Очевидно, вместо изображения может быть любой массив. Вторым аргументом функции – **order** – управляет памятью при копировании. По умолчанию задается значение 'K': как можно более точный подбор. Возвращаемое значение функции – копия массива (в нашем случае – изображения).

Допустим, необходимо не скопировать изображение, а создать пустое изображение определенного размера. NumPy позволяет создать массив заданного размера, инициализированный нулями. Для этого можно воспользоваться функцией **zeros()**.

```
res = np.zeros(
    shape,          # размер массива
    dtype=np.float64, # тип данных
    order='C',      # тип хранения многомерных данных
)
```

Первый аргумент – **shape** – определяет размер массива. В случае одномерного массива задается целое число, в иных случаях – кортеж целых чисел. Для изображения будут типичны два варианта: (h, w) и (h, w, ch) , где h – высота изображения (число строк), а w – ширина изображения (число столбцов). Два параметра подойдут для одноканального изображения. В случае трехканального изображения можно указать значение ch – количество каналов – 3. Вторым аргументом – **dtype** – определяет тип данных элементов, которыми заполняется массив. По умолчанию выбираются числа с плавающей точкой 64 бит. Изображения в оттенках серого и RGB имеют глубину каждого канала 8 бит. Для них нужно указать значение `np.uint8`. Последний аргумент – **order** – определяет, как хранятся многомерные данные.

В данном случае все элементы массива заполнены нулями. Нули будут отображаться и в RGB, и в оттенках серого черными пикселями. Создать массив заданного размера, заполненный любыми числами, можно с помощью функции **full()**.

```
res = np.full(  
    shape,    # размер массива  
    val,      # значение для заполнения  
    dtype,    # тип данных  
    order     # тип хранения многомерных данных  
)
```

Аргументы **shape**, **dtype** и **order** принимают те же значения, что и в функции **zeros()**. Значения по умолчанию те же. В аргумент **val** передается значение, которое должны принять элементы массива. Это должно быть одно число или массив, и значение должно соответствовать типу данных.

Иногда бывает полезно не только выполнять операции над массивами, но и извлекать информацию об изображении. Например, атрибут **shape** позволяет определить размерность массива – ширину и высоту изображения, а также число каналов.

```
res = img.shape
```

В **res** будет храниться кортеж из двух или трех элементов. Два элемента будут в случае, если изображение хранится в оттенках серого. Так как третий элемент – число каналов, для RGB изображений он будет равен 3.

Так как NumPy позволяет выполнять различные операции над массивами, можно применить это знание для быстрого изменения данных каждого пикселя. Например, одинаковые операции можно выполнять сразу над всем массивом:

```
img = 255 - img
```

Если **img** – изображение, прочитанное в оттенках серого, то такая операция инвертирует изображение. Пиксели со значением яркости 0 будут иметь значение яркости 255, со значением 1 – 254, 2 – 253 и так далее. Если **img** – изображение, прочитанное в цвете, то будут инвертированы значения каждого канала. Доступ к одному пикселю осуществляется так же, как при работе с любыми массивами. При работе с изображениями можно также получить доступ ко всем элементам строки или столбца с помощью символа ':'. Например, присвоим значениям канала *G* для всех пикселей значения канала *B* для тех же пикселей:

```
img[:, :, 1] = img[:, :, 0]
```

Для всех значений строк и столбцов будет выполнена одинаковая операция. Похожий инструментарий для работы с матрицами есть также в пакете Matlab.

1.2.2 Редактирование изображений и видеопотока в OpenCV

При чтении изображения можно указать, в какой цветовой модели его нужно считать. А что, если в задаче нужно иметь два изображения: цветное – допустим, RGB – и в оттенках серого? Очевидный ответ – прочитать изображение еще раз, указав иной параметр. Однако это не всегда удобно. Бывают также случаи, когда дважды получить одно и то же изображение невозможно: например, при работе с отдельными кадрами видеопотока. В таком случае можно воспользоваться функцией, которая позволяет конвертировать изображение в нужный формат – **cvtColor()**.

```
img2 = cv2.cvtColor(  
    img,      # входное изображение  
    code     # код преобразования  
)
```

Первым аргументом передается входное изображение – **img**. Во второй аргумент – **code** – передается код преобразования. При работе с изображениями в RGB нужно помнить, что в OpenCV возможны два порядка каналов: RGB и BGR. Для двух этих вариантов существуют разные коды. Например, для перевода изображения, прочитанного функцией **imread()** с флагом **cv2.IMREAD_COLOR**, в изображение в оттенках серого нужно указать код **cv2.COLOR_BGR2GRAY**.

В OpenCV также можно работать с каналами в виде отдельных массивов. Изображение с несколькими каналами можно разбить по каналам с помощью функции **split()**.

```
b, g, r = cv2.split(img)
```

Здесь **img** – изображение, прочитанное функцией **imread()** с флагом **cv2.IMREAD_COLOR**. Тогда оно состоит из трех каналов: синего, зеленого и красного. Функция **split()** разбила изображение на три одноканальных изображения. Первое изображение – **b** – содержит те же значения яркости пикселей, которые значились для синего канала исходного изображения. Соответственно, **g** и **r** – изображения с яркостью пикселей как у зеленого и красного каналов исходного изображения.

Для функции разделения каналов есть парная функция, выполняющая совмещение каналов: **merge()**.

```
img_n = cv2.merge([g, b, r])
```

В аргумент функции необходимо передать исходные изображения в массиве. Здесь **b**, **g** и **r** – одноканальные изображения, полученные с помощью функции **split()**. Так как в функцию **merge()** они передаются в порядке **g**, **r**, **b**, в результате получится новое изображение – **img_n**. При этом значения яркости для синего канала заменятся на значения яркости зеленого и наоборот.

Часто в задачах технического зрения для проверки работы отдельных алгоритмов и отладки необходимо рисовать на изображениях простые геометрические фигуры: прямоугольники, круги, линии и т. д. В OpenCV предусмотрен ряд функций для таких действий. Например, нарисовать линию можно с помощью функции **cv2.line()**.

```
cv2.line(  
    img,                # входное изображение  
    pt1,                # координаты начала  
    pt2,                # координаты конца  
    color,              # цвет  
    thickness=1,        # толщина линии  
    lineType=cv2.LINE_8, # тип линии  
    shift=0             # дробные биты  
)
```

Первым аргументом – **img** – передается изображение, на котором нужно рисовать. Эта функция, как и другие функции для рисования, изменяет исходное изображение! Следующие два элемента – **pt1** и **pt2** – координаты начала и конца отрезка, соответственно. Каждый элемент принимает кортеж из двух целых чисел – (x, y). Начало координат при этом расположено в верхнем левом углу. Ось X направлена слева направо, а ось Y – сверху вниз. Следующий элемент – **color** – задает цвет линии. В него нужно передать кортеж из трех элементов: значения синего, зеленого и красного каналов: (b, g, r). Значения должны быть целыми числами в диапазоне от 0 до 255.

Остальные аргументы являются необязательными, так как их значения заданы по умолчанию. Для всех функций рисования их толкование будет одинаковым. В аргумент **thickness** передается толщина линии в пикселях. Аргумент **lineType** определяет тип линии, которой будет нарисована фигура. Возможные типы линий описаны в документации. Аргумент **shift** указывает на количество дробных битов в точечных координатах.

Для рисования окружностей можно воспользоваться функцией **circle()**.

```
cv2.circle(  
    img,                # входное изображение  
    center,             # координаты центра окружности  
    radius,             # радиус  
    color,              # цвет линии  
    thickness=1,        # толщина линии  
    lineType=cv2.LINE_8, # тип линии  
    shift=0             # дробные биты  
)
```

Первый аргумент – **img** – тоже принимает изображение, на котором будет нарисована окружность. Второй аргумент – **center** – принимает кортеж из двух элементов (x, y). Это координаты в пикселях точки – центра окружности. Аргумент **radius** принимает число – длину радиуса. Остальные аргументы те же, что и при рисовании линий.

Для рисования прямоугольников в OpenCV есть функция **rectangle()**.

```
cv2.rectangle(  
    img,                # входное изображение  
    pt1,               # коорд. левого верхнего угла  
    pt2,               # коорд. правого нижнего угла  
    color,              # цвет линии  
    thickness=1,        # толщина линии # тип линии  
    lineType=cv2.LINE_8, # дробные биты  
    shift=0  
)
```

Аргументы **img**, **color**, **thickness**, **lineType** и **shift** принимают те же данные, что и в функциях рисования линий и окружностей. Аргумент **p1** – координаты левого верхнего угла прямоугольника. И аргумент **pt2** – координаты правого нижнего угла прямоугольника. Координаты также задаются кортежем из двух чисел – (x, y).

Помимо отрисовки геометрических фигур, полезно также иметь возможность надписать некоторые элементы изображения. Для таких целей служит функция **putText()**.

```
cv2.putText(  
    img,                # исходное изображение  
    text,               # что нужно написать  
    pt,                # координата левого нижнего угла  
    fontFace,          # шрифт  
    fontScale,         # масштабирование шрифта  
    color,             # цвет  
    thickness=1,       # толщина линии  
    lineType=cv2.LINE_8, # тип линии  
    bottomLeftOrigin=False # отражение  
)
```

Первым аргументом передается входное изображение – **img**. В результате работы этой функции оно будет перезаписано! Во второй аргумент – **text** – передается надпись в виде строки. Аргумент **pt** задает координату левого нижнего угла прямоугольника, в котором будет содержаться надпись. Координаты передаются кортежем из двух элементов – (x, y). В аргумент **fontFace** передается шрифт. В OpenCV есть набор шрифтов, которые можно использовать при создании надписей. Все варианты описаны в перечислении (enum) *HersheyFonts*. Его описание можно найти в документации. Следующий аргумент – **fontScale** – определяет масштабирующий коэффициент, который будет применен к исходному размеру выбранного шрифта. Аргументы **color**, **thickness** и **lineType** для этой функции совпадают с теми же аргументами функций для рисования геометрических фигур. Аргумент **bottomLeftOrigin** определяет, как расположена надпись относительно изображения. Если установить его в *True*, надпись будет отражена относительно горизонтали.

Все операции по работе с отдельными изображениями можно применять к кадрам видеопотока. Все операции по работе с видео должны покадрово выполняться в цикле. За одну итерацию цикла обрабатывается один кадр. Основные операции в цикле для работы с видеопотоком:

- 1) прочитать один кадр;
- 2) отобразить один кадр, если необходимо;
- 3) ждать нажатия на клавишу в течение нескольких миллисекунд.

Операции по отображению кадра и ожиданию нажатия на клавишу выполняются с помощью уже знакомых функций – **imshow()** и **waitKey()**. Однако при

работе с видео в аргумент **waitKey()** обязательно передавать число больше нуля, но недостаточно большое, чтобы возникала потеря большого количества кадров.

Для захвата видео из файлов, с камеры или работы с последовательностью изображений в OpenCV предназначен класс **VideoCapture**. Класс имеет три варианта конструктора. При чтении данных с камеры первым аргументом нужно передать индекс камеры: 0, если камера одна. Вторым аргументом можно принудительно указать реализацию чтения. Чтение отдельного кадра выполняется методом **read()**, который вызывается у экземпляра класса **VideoCapture**. Приведем пример скрипта для чтения видео.

```
# импорт библиотеки
import cv2.cv2 as cv2

# создание экземпляра класса для захвата видео с камеры 0
cap = cv2.VideoCapture(0)
while(True):
    # Чтение одного кадра
    ret, frame = cap.read()

    # Отображение кадра
    cv2.imshow('frame', frame)
    # Ожидание нажатия на ESC в течение 1 мс
    if cv2.waitKey(1) == 27:
        break
```

Метод **read()** возвращает два параметра: статус чтения и сам кадр. Если кадр не был прочитан, первое возвращаемое значение будет *False*. Здесь описаны только базовые операции по работе с захватом видеопотока. Более подробно с возможностями по работе с видео и последовательностью изображений можно познакомиться в документации к классу **VideoCapture**.



СПбГЭТУ «ЛЭТИ»
ПЕРВЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ

Фильтры: теория и практика

2.1 ОСНОВЫ ПРИМЕНЕНИЯ ФИЛЬТРОВ К ИЗОБРАЖЕНИЯМ

Модуль 2.1 посвящен общим вопросам применения фильтров к изображениям. С этого момента изображения будут рассматриваться не просто как двумерный или трехмерный массив пикселей, а как информация, представленная в графическом виде.

В первой части будут рассмотрены базовые понятия применения фильтров к изображениям, такие как «*фильтр*», «*ядро*», «*свертка*», «*якорь*». Также будет рассмотрено, как именно происходит применение фильтров к изображениям с точки зрения вычислений и на что влияет содержание и размер фильтра. Во второй части модуля будут рассмотрены особенности обработки изображений на границах: идея создания рамок вокруг изображений, возможные варианты рамок и средства OpenCV, которые позволяют работать с рамками.

2.1.1 О фильтрах, ядрах и свертке

Прежде чем приступить к изучению высокоуровневых методов обработки изображений и применению их на практике, необходимо познакомиться с базовыми понятиями из этой области. Для начала определим, что такое фильтр в целом.

Пусть мы имеем некоторое изображение I_1 . Наша задача – изменить это изображение и получить новое: I_2 . Причем каждый пиксель нового изображения $I_2(x, y)$ должен получать свое значение в зависимости от значения пикселей некоторой окрестности пикселя с теми же координатами на старом – $I_1(x, y)$.

Получается, что мы имеем алгоритм, который получает на вход изображение I_1 и возвращает новое изображение I_2 , при этом пересчитав значения каждого пикселя $I_2(x, y)$ путем комбинирования пикселей окрестности $I_1(x, y)$. Способ комбинирования пикселей и размер окрестности – основа данного алгоритма. Сочетание этих двух элементов и является **фильтром**.

В контексте обработки изображений фильтр также иногда называют **ядром**. Здесь эти термины взаимозаменяемы. Разница будет заключаться только в том, что термин «*фильтр*» чаще употребляется в контексте обработки сигналов, а термин «*ядро*» – в математике.

Итак, пиксели изображения I_2 зависят от пикселей изображения I_1 . Если эта зависимость линейная, то изображение I_2 получено из изображения I_1 с применением **линейного** фильтра. И, очевидно, если зависимость нелинейная, то для получения изображения I_2 к изображению I_1 применялся **нелинейный** фильтр.

Многие фильтры, которые часто применяются при обработке изображений, являются именно линейными фильтрами. Поэтому подробнее разберем, какие вычисления производятся при применении линейного фильтра. Если к изображению I_1 применяется линейный фильтр, то значение каждого пикселя изображения $I_2(x, y)$ можно выразить в виде взвешенной суммы окрестностей пикселя $I_1(x, y)$. Это можно записать в виде формулы (2.1):

$$I_2(x, y) = \sum_{i=0}^{i < h} \sum_{j=0}^{j < w} K_{i,j} \cdot I_1(x + i, y + j), \quad (2.1)$$

где K – фильтр, представленный в виде матрицы; h – высота матрицы (число строк); w – ширина матрицы (число столбцов); I_1 и I_2 – входное и выходное изображения; (x, y) – координаты пикселя, относительно которого выполняются вычисления.

Если фильтр можно записать таким образом, то он будет линейным. Линейные фильтры также называют **свертками**. В контексте технического зрения термин «свертка» иногда употребляется по отношению к любым фильтрам вне зависимости от того, линейные они или нет.

Для лучшего понимания того, что такое фильтр, удобно представить его графически в виде массива. На рис. 2.1 изображено графическое представление двух типичных фильтров размера 5 на 5 пикселей.

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

a

1	4	7	4	1
4	16	26	16	4
7	26	41	26	7
4	16	26	16	4
1	4	7	4	1

б

Рисунок 2.1 – Пример фильтра:

a – прямоугольный фильтр; *б* – фильтр Гаусса

И у прямоугольного фильтра, и у фильтра Гаусса на рис. 2.1 есть особенный элемент – **якорная точка**. Здесь это центральные элементы, которые на рисунке обведены в круг. Якорная точка определяет то, как именно фильтр будет накла-

дываться на изображение. Мы помним, что фильтр применяется к каждому пикселю изображения. Якорная точка – это та точка фильтра, которая накладывается на точку $I_1(x, y)$ при расчете значения пикселя $I_2(x, y)$. Если якорная точка находится посередине, при расчете нового значения $I_2(x, y)$ учитывается одинаковое число пикселей во всех направлениях окрестности.

На рис. 2.1 представлены **ненормированные** варианты прямоугольного фильтра и фильтра Гаусса. При применении таких фильтров значение пикселей нового изображения будет увеличиваться. Причем чем больше размер ядра, тем больше пикселей и коэффициентов будут участвовать в суммировании и тем больше будет увеличение. Существует также вариант применения нормированных ядер. В этом случае все значения матрицы будут делиться на сумму элементов матрицы. Например, сумма элементов ненормированного фильтра Гаусса размера 5 на 5 пикселей равна 273. При расчете $I_2(x, y)$ пиксель $I_1(x, y)$ будет умножен на $41/273$, пиксель $I_1(x+1, y)$ – на $26/273$, пиксель $I_1(x+2, y)$ – на $7/273$ и так далее.

Теперь разберем, как изменится изображение, если применить фильтр не к одному пикселю, а к каждому пикселю изображения. На рис. 2.2 изображено применение некоторого фильтра размера 3 на 3 к изображению 5 на 5 пикселей.

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

a

0	1	2
2	2	0
0	1	2

б

12	12	17
10	17	19
9	6	14

в

Рисунок 2.2 – Применение фильтра:

a – исходное изображение; *б* – фильтр; *в* – результат

К исходному изображению 5 на 5 пикселей применяется фильтр размером 3 на 3. Якорная точка в нашем случае – средняя точка фильтра. Будем накладывать фильтр на изображение, начиная с левого верхнего угла. При этом нужно наложить фильтр так, чтобы вся матрица покрывала существующие пиксели изображения. На рис. 2.2, *a* продемонстрировано, как будет наложен фильтр на первом шаге. Если верхний левый пиксель изображения имеет координаты (0, 0), то якорная точка для фильтра размером 3 на 3 будет наложена на пиксель с ко-

ординатами (1, 1). Новое значение для данного пикселя изображения будет равно 12. Оно выделено на рис. 2.2, в. Очевидно, что значение для точки (0, 0) исходного изображения не может быть пересчитано: за границами изображения отсутствуют пиксели, на которые будет наложен фильтр. То же верно для всех пикселей исходного изображения, которые примыкают к границам. Очевидно, что простое применение фильтра без дополнительных мер приводит к уменьшению размера изображения. При этом чем больше фильтр, тем меньше будет новое изображение.

2.1.2 Добавление рамок к изображениям

Для того чтобы иметь возможность при применении фильтра к изображению не уменьшать его размер, можно дополнить изображение пикселями на границах – создать рамку. На рис. 2.3 приведен пример добавления рамки к изображению.

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

а

?	?	?	?	?	?	?
?	3	3	2	1	0	?
?	0	0	1	3	1	?
?	3	1	2	2	3	?
?	2	0	0	2	2	?
?	2	0	0	0	1	?
?	?	?	?	?	?	?

б

Рисунок 2.3 – Процесс добавления рамки к изображению:

а – исходное изображение; *б* – изображение с рамкой

На изображении 2.3, б к исходному изображению добавлена рамка шириной в 1 пиксель. Единственно верного способа задания значения пикселям, которые принадлежат рамке, не существует. В OpenCV, например, существует несколько разных способов выбора значений как в зависимости от содержимого изображения, так и нет. Вне зависимости от того, какие значения заполняют область рамки, она позволяет применять фильтры, совмещая якорную точку с пикселем на границе. На рис. 2.4 изображен пример наложения фильтра размера 3 на 3 на изображение с рамкой.

0	1	2
2	2	0
3	1	2

a

?	?	?	?	?	?	?
?	3	3	2	1	0	?
?	0	0	1	3	1	?
?	3	1	2	2	3	?
?	2	0	0	2	2	?
?	2	0	0	0	1	?
?	?	?	?	?	?	?

б

Рисунок 2.4 – Применение фильтра:

a – фильтр размера 3 на 3; *б* – применение фильтра к пикселю (0, 0)

На рис. 2.4, *б* показано, как накладывается фильтр на изображение с якорем. Фильтр, изображенный на рис. 2.4, *a*, имеет размер 3 на 3. Таким образом, чтобы совместить его якорную точку с пикселем изображения с координатами (0, 0), нужно дополнить изображение рамкой шириной в 1 пиксель.

Теперь разберемся, как можно задать значения пикселям, которые принадлежат рамке. Так как здесь практические вопросы рассматриваются в контексте применения инструментария, предоставляемого библиотекой OpenCV, разберем те варианты рамок, которые реализованы в данной библиотеке.

На первом шаге определим, как именно можно создать рамку. Для этой задачи можно воспользоваться функцией **copyMakeBorder()**.

```
img2 = cv2.copyMakeBorder(
    img,                # входное изображение
    top,                # пиксели сверху
    bottom,             # пиксели снизу
    left,               # пиксели слева
    right,              # пиксели справа
    bType=cv2.BORDER_DEFAULT, # тип рамки
    value               # значение для cv2.BORDER_CONSTANT
)
```

В первый аргумент функции – **img** – передается входное изображение, для которого нужно построить рамку. Следующая четверка аргументов – **top**, **bottom**, **left** и **right** – определяет, сколько пикселей будет составлять рамка для

верхней, нижней, левой и правой границ, соответственно. Последний аргумент – **bType** – определяет тип рамки. Возвращаемое значение функции – изображение, дополненное рамкой в соответствии с указанным типом рамки и количеством пикселей рамки для каждой стороны.

От того, какой тип рамки выбран, зависит то, какие значения будут принимать пиксели, которыми дополняется изображение. Подробно описание типов границы и особенностей их применения изложено в документации к OpenCV. Приведем наиболее примечательные типы рамки. По умолчанию аргумент **bType** принимает значение `cv2.BORDER_DEFAULT`. В таком случае используется тот же тип дополнения рамки, что и для варианта `cv2.BORDER_REFLECT_101`. У такого способа дополнения рамки есть еще один эквивалент – `cv2.BORDER_REFLECT101`. Пример построения рамки для такого изображения приведен на рис. 2.5

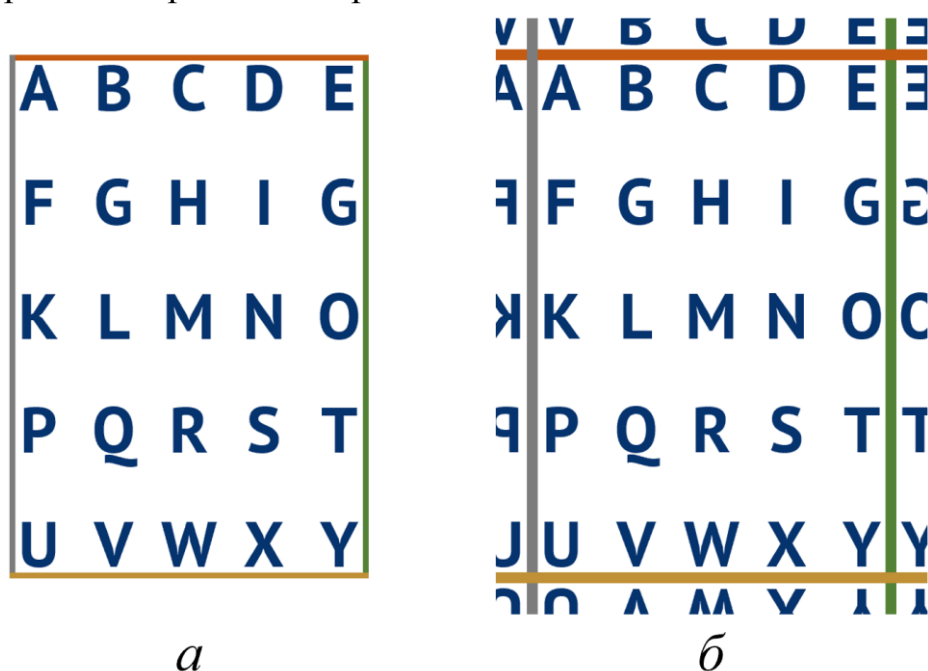


Рисунок 2.5 – Рамка по умолчанию:

a – исходное изображение; *б* – изображение с рамкой типа `cv2.BORDER_DEFAULT`, `cv2.BORDER_REFLECT101`, `cv2.BORDER_REFLECT_101`

Этот тип рамки заполняет пиксели границы по стороне, располагая изображение зеркально относительно этой стороны. При этом пиксель на границе исходного изображения не дублируется. В этом примере, как и во всех последующих, для построения рамки по каждой стороне выбрана ширина в 50 пикселей. На эти три типа рамки похож еще один – `cv2.BORDER_REFLECT`. Визуально результат практически не отличается. Единственное отличие такого типа заклю-

чается в том, что при `cv2.BORDER_REFLECT` граничный пиксель изображения тоже отражается и участвует в рамке.

Еще три примечательных способа построить границу – применить тип построения рамки `cv2.BORDER_WRAP`, `cv2.BORDER_CONSTANT` или `cv2.BORDER_REPLICATE`. Они представлены на рис. 2.6.

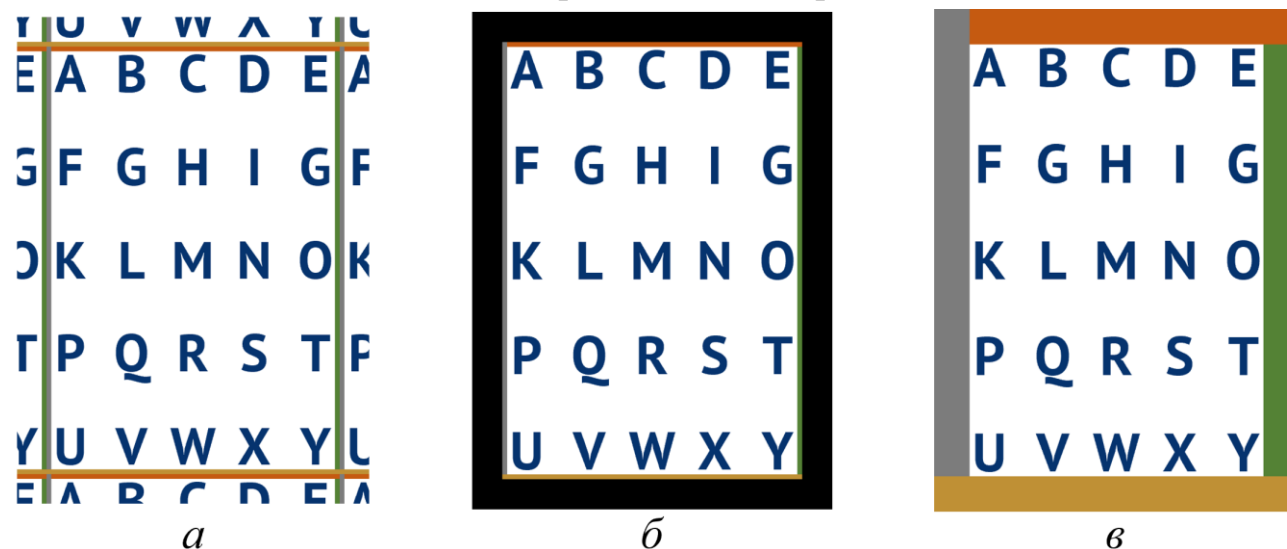


Рисунок 2.6 – Рамка по умолчанию:

а – рамка `cv2.BORDER_WRAP`; б – рамка `cv2.BORDER_CONSTANT`; в – рамка `cv2.BORDER_REPLICATE`

Первый вариант – `cv2.BORDER_WRAP` – дополняет изображение рамкой, в которой участвуют пиксели с противоположной стороны изображения. Третий вариант – `cv2.BORDER_REPLICATE` – повторяет пиксели, которые находятся на соответствующей границе изображения. Вариант `cv2.BORDER_CONSTANT` заполняет рамку пикселями с одинаковым значением, которое можно передать в аргумент **value**. Если аргумент не задан, то рамка будет заполнена пикселями с нулевыми значениями каналов.

В случае если изображение, с которым мы работаем, является частью некоторого большого изображения, его можно назвать областью интереса – **ROI**, *region of interest*. Когда рамка строится для ROI, функция построения рамки будет пытаться использовать внешние для этой области пиксели, чтобы построить границу. Такое поведение алгоритма не всегда подходит. Чтобы отключить эту опцию и использовать один из стандартных способов, нужно дополнительно указать тип границы `cv2.BORDER_ISOLATED`.

2.2 ПОРОГОВЫЕ ПРЕОБРАЗОВАНИЯ

Модуль 2.2 посвящен вопросам преобразования значений пикселей по признаку яркости. В техническом зрении существуют такой инструмент обработки изображений, как пороговые преобразования. Они предназначены для выполнения определенных операций над яркостью пикселей, суть которых зависит от того, больше или меньше значение яркости пикселя, чем у некоторого порога. В первой части модуля рассматриваются различные типы преобразований, которые могут быть применены к отдельному пикселю. Во второй части модуля приводятся три возможных подхода к применению таких преобразований: один порог для всех пикселей, задаваемый вручную; один порог для всех пикселей, рассчитанный автоматически; адаптивный порог – уникальный порог для каждого пикселя в зависимости от его окрестности.

2.2.1 Основная информация о пороговых преобразованиях

На этапе проектирования систем технического зрения, когда каждый пиксель изображения несет какую-то информацию, важно иметь возможность исключать из обработки те пиксели, которые не несут важной информации для текущей задачи. Например, разберем задачу, когда робот должен пересечь дорогу по пешеходному переходу. С помощью камеры робот получает изображение в цвете. Его задача – определить, есть ли на изображении пешеходный переход, и если есть, то найти его границы, чтобы определить полосу, внутри которой можно двигаться. Однако на таком изображении будет множество лишней информации, которую можно исключить, чтобы уменьшить объем вычислений.

Первое, что можно сделать – уменьшить число каналов, конвертировав изображение в RGB в изображение в оттенках серого. На рис. 2.7 приведен пример такого преобразования для возможного варианта кадра с камеры.

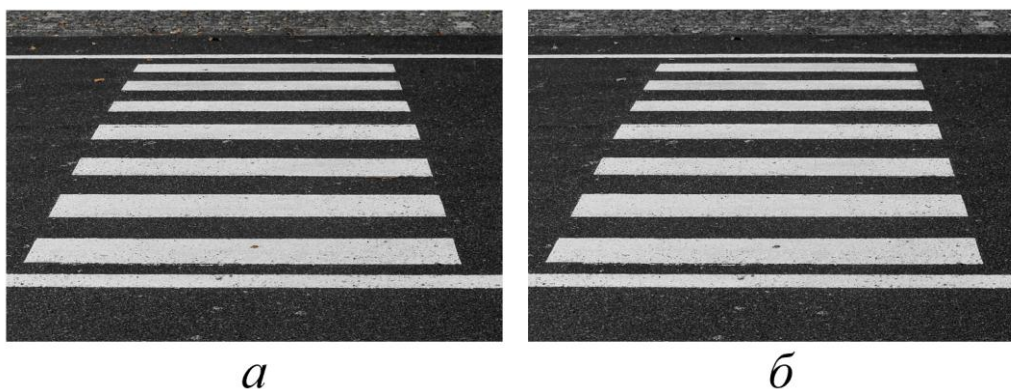


Рисунок 2.7 – Уменьшение числа каналов на изображении:
а – исходное изображение; б – изображение в оттенках серого

С визуальной точки зрения различие незначительное. Это связано с тем, что камера направлена вниз и захватывает только поверхность, по которой движется робот. В данном случае в кадр попадают только дорожное полотно, которое изначально серое, разметка – изначально белая, и небольшое число посторонних предметов в цвете – листья. Поэтому уменьшение каналов практически не меняет изображение с точки зрения человека. Однако с точки зрения информации, которая хранится в массиве изображения, изменение существенное. Изображение на рис. 2.7, *а* – это трехмерный массив, который для каждого пикселя хранит значения трех каналов. А изображение на рис. 2.7, *б* – уже двумерный массив, где для каждого пикселя хранится значение только одного канала. Это позволило уменьшить объем информации в 3 раза.

Если перед роботом стоит задача именно определить наличие пешеходного перехода и его ориентацию в пространстве, то можно еще сильнее уменьшить объем информации на изображении. Рассмотрим внимательно пиксели, которые принадлежат дорожному полотну. Все они имеют свой оттенок. То есть у этих пикселей значения яркости разные, хотя и находятся в одном диапазоне. То же можно сказать и о пикселях, которые относятся к разметке. Большинство пикселей имеют довольно высокие значения яркостей, но, тем не менее, не максимальные. Получается, что и дорожное полотно, и разметка содержат информацию о полутонах, которая в контексте этой задачи является лишней.

Поэтому для данной задачи можно изменить значения пикселей, руководствуясь значением яркости каждого пикселя. Очевидно, что яркость пикселей полотна сильно отличается от яркости пикселей разметки. И между значениями яркости, характерной для этих элементов, можно провести некоторый **порог**. Значит, найдется такое значение яркости, для которого яркость пикселей полотна будет меньше порога, а яркость пикселей разметки – больше порога. Тогда всем пикселям, значение яркости которых меньше порога, можно присвоить минимальное значение яркости, а остальным – максимальное. Посмотрим, как будет выглядеть такое преобразование. На рис. 2.8 приведен пример двух изображений – исходное изображение в оттенках серого и изображение, к которому применили описанное выше преобразование. Если внимательно рассмотреть второе изображение – рис. 2.8, *б*, то можно заметить, что часть пикселей фона получила максимальное значение яркости (белый цвет), а часть пикселей разметки – минимальное (черный цвет). Для наших целей этой погрешностью можно пренебречь.

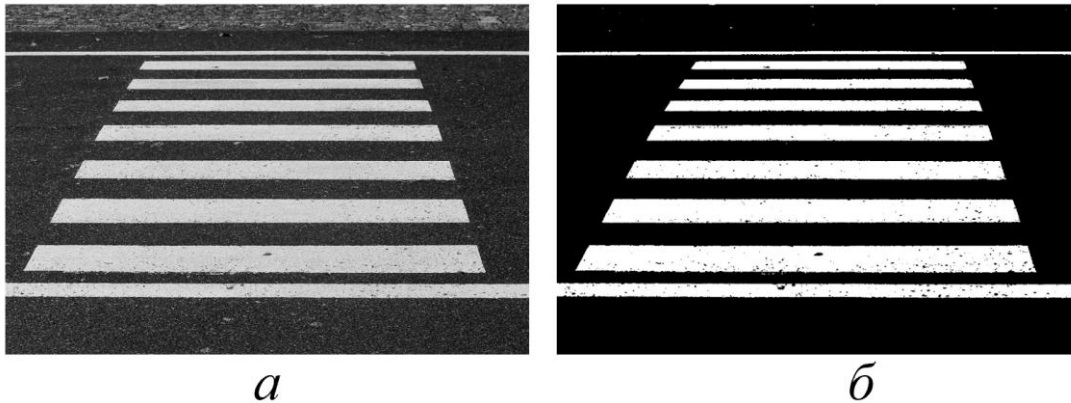


Рисунок 2.8 – Удаление полутонов:

а – изображение в оттенках серого; *б* – результат изменения яркости пикселей

С точки зрения математики для каждого пикселя (x, y) изображения I выполнялось следующее преобразование, которое можно описать формулой 2.2:

$$I_2(x, y) = \begin{cases} \text{действие 1, } I_1(x, y) > T \\ \text{действие 2, } I_1(x, y) \leq T \end{cases} \quad (2.2)$$

где $I_1(x, y)$ – пиксель на исходном изображении; $I_2(x, y)$ – пиксель на изображении, содержащем результат; T – порог яркости; *действие 1* и *действие 2* – условные обозначения двух вариантов изменения значения яркости пикселя. Такое преобразование называется **пороговым**.

Получается, что в результате применения порогового преобразования все пиксели можно будет разделить на две группы. В одной группе будут пиксели, значение яркости которых было изменено *действием 1*. И во второй группе будут пиксели, для которых значение яркости было изменено *действием 2*. Для случая, представленного на рис. 2.8, будут использованы значения по формуле (2.3):

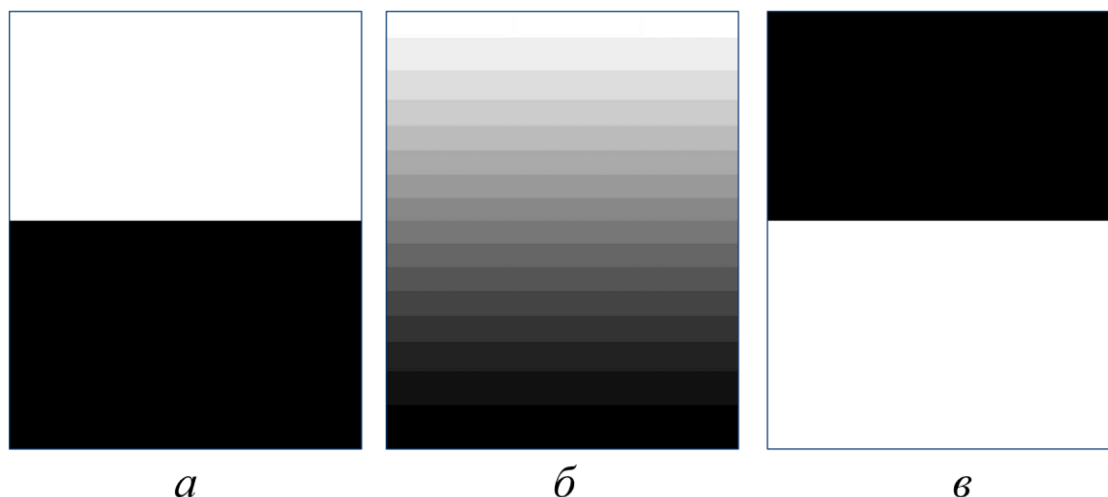
$$I_2(x, y) = \begin{cases} 255, & I_1(x, y) > 170 \\ 0, & I_1(x, y) \leq 170 \end{cases} \quad (2.3)$$

где $I_1(x, y)$ – пиксель на изображении, представленном на рис. 2.8, *а*; $I_2(x, y)$ – пиксель на изображении, представленном на рис. 2.8, *б*.

Существует несколько вариантов пар действий, которые могут быть выполнены над пикселями. Разберем варианты пороговых преобразований, которые реализованы в OpenCV.

Во-первых, можно осуществить **бинаризацию** изображения. **Бинарным** будем считать изображение, яркость пикселей которого принимает только два значения – 0 и 255. Очевидно, что для хранения данных о яркости пикселя такого изображения все еще требуется 8 бит. Но так как возможных значений ярко-

сти только два – минимальное и максимальное, можно назвать изображение бинарным. Один из вариантов бинаризации как раз представлен на рис. 2.8, б. На практике можно сделать бинаризацию двумя способами – рис. 2.9.



а

б

в

Рисунок 2.9 – Бинаризация пороговым преобразованием:

а – прямое преобразование; б – исходное изображение; в – инверсное преобразование

В случае с рис. 2.9, *а* преобразование значения яркости пикселей выполнялось в соответствии с формулой 2.2. *Действие 1* – присваивание яркости пикселя значения 0, *действие 2* – присваивание значения 255. В данном случае порог T имеет значение 127. Для рис. 2.9, *в* ситуация противоположная: *действие 1* и *действие 2* поменялись местами. Попробуем записать формулу 2.2 алгоритмически:

ЕСЛИ яркость $> T$ **ТО**

действие 1

ИНАЧЕ

действие 2

КОНЕЦ

Очевидно, что в варианте алгоритма, с помощью которого получилось инвертированное бинарное изображение – рис. 2.9, *в*, в условии поменялся знак. Если раньше условие звучало как «яркость $> T$ », то для инверсии использовалось условие «яркость $< T$ ».

Инверсные методы существуют, так как пороговые преобразования обычно используются для предварительной обработки изображений. Многие алгоритмы выполняются только для точек, значение яркости которых больше 0. Следовательно, и в случае прямой, и в случае инверсной бинаризации в каждой группе

яркости будут одни и те же пиксели, имеет значение, какие пиксели будут принимать значение 0, а какие – нет.

Еще два парных метода порогового преобразования позволяют обнулить только те пиксели, которые не удовлетворяют условию, а яркость остальных оставить без изменений – рис. 2.10.

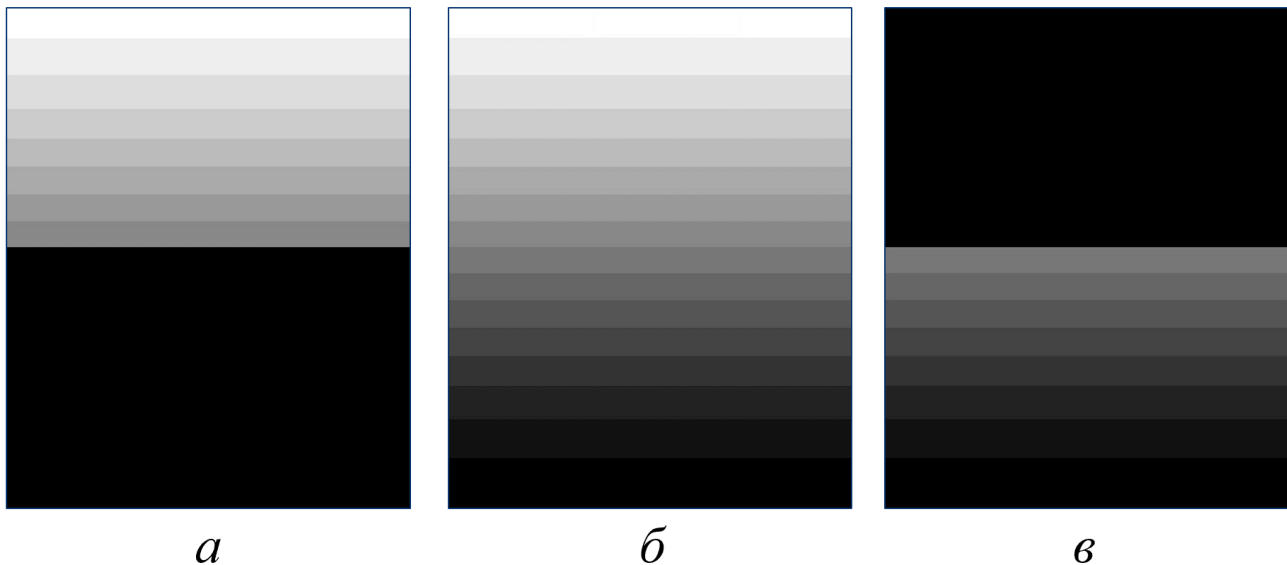


Рисунок 2.10 – Обнуление яркости части пикселей:

а – прямое обнуление; б – исходное изображение; в – инверсное обнуление

Такие методы будут полезны в случае, когда для последующей обработки все еще важна информация о яркости пикселей, но пиксели с яркостью из некоторого диапазона должны быть исключены из рассмотрения. При этом диапазон будет либо от T до максимума, либо от T до минимума. Для лучшего понимания посмотрим на то, как менялась яркость пикселей для рис. 2.10, *а*: формула 2.4.

$$I_2(x, y) = \begin{cases} I_1(x, y), & I_1(x, y) > T \\ 0, & I_1(x, y) \leq T \end{cases}, \quad (2.4)$$

Очевидно, что для случая на рис. 2.10, *в* знаки в формуле 2.4 должны быть изменены.

Существует еще один метод порогового преобразования, который, можно сказать, является «парным» для инверсного обнуления – ограничение яркости. В случае с данным методом пиксели, которые не прошли порог, будут принимать значение яркости, указанное как пороговое. Сравнение инверсного обнуления яркости и ограничения яркости представлено на рис. 2.11.

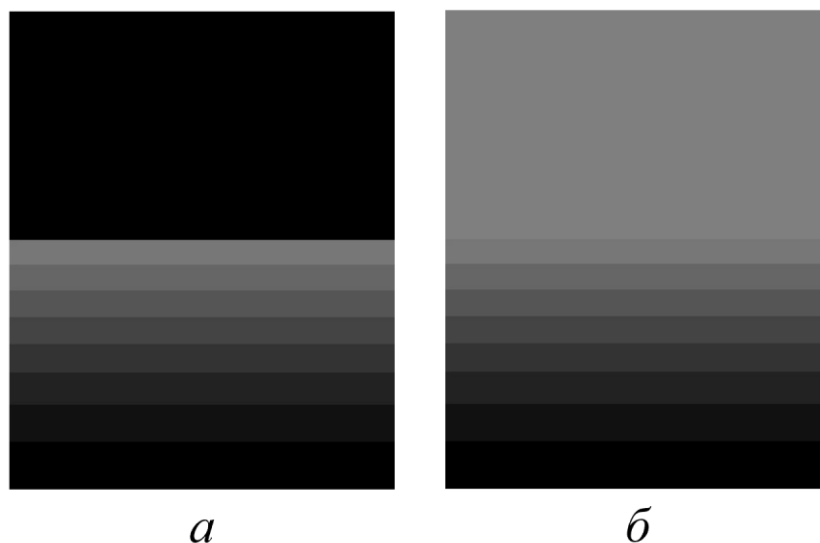


Рисунок 2.11 – Сравнение преобразований:

а – инверсное обнуление яркости; б – ограничение яркости

Хотя изменяются пиксели с той же яркостью, метод на рис. 2.11, *а* все еще является инверсным, тогда как метод на рис. 2.11, *б* – прямой. Парного инверсного метода для него не существует, но на практике можно комбинировать различные методы.

2.2.2 Влияние пороговых преобразований на изображения

При проектировании систем технического зрения с помощью OpenCV доступны два вида преобразований: с использованием одинакового порога для всех пикселей и с использованием разных значений порогов, которые рассчитываются для каждого пикселя в отдельности. При использовании одинакового порога также возможны два варианта задания порога: указать вручную или посчитать автоматически.

В OpenCV для применения пороговых преобразований, где для всех пикселей изображения используется один и тот же порог, можно воспользоваться функцией **threshold()**.

```
res = cv2.threshold(
    img,          # входное изображение
    threshold,    # порог
    maxVal,       # максимально возможное значение яркости
    type          # тип преобразования
)
res = threshold_new, new_img
```

Первый аргумент – **img** – принимает изображение, для которого нужно применить пороговое преобразование. Во второй аргумент – **threshold** – передается значение порога, с которым будет сравниваться яркость пикселей. В третий аргумент – **maxVal** – передается значение, которое будет присвоено пикселям, значение яркости которых больше порогового значения, для преобразований, которые выполняют бинаризацию изображения. И в последний аргумент – **type** – передается тип преобразования.

Функция возвращает два значения. Первое возвращаемое значение – **threshold_new** – хранит значение порога, которое было рассчитано автоматически. В случае, если порог задавался вручную, это возвращаемое значение будет равно значению, которое передавалось в аргумент **threshold**. Второе возвращаемое значение – **img_n** – новое изображение. При применении этой функции исходное изображение, которое передается в аргумент **img**, не меняется.

Для ручного варианта задания порога подходят пять вариантов значений, которые можно передать в аргумент **type**:

- 1) **cv2.THRESH_BINARI** – бинаризация изображения (рис. 2.9, а);
- 2) **cv2.THRESH_BINARY_INV** – инверсная бинаризация изображения (рис. 2.9, в);
- 3) **cv2.THRESH_TOZERO** – обнуление яркости части пикселей изображения (рис. 2.10, а);
- 4) **cv2.THRESH_TOZERO_INV** – инверсное обнуление яркости части пикселей изображения (рис. 2.10, в);
- 5) **cv2.THRESH_TRUNC** – ограничение яркости части пикселей изображения (рис. 2.11, б).

Автоматический расчет порога яркости пикселей реализован в двух вариантах: методом треугольника и методом Оцу. Оба варианта основаны на идее выбора некоторого близкого к среднему значения для распределения яркостей разного уровня на конкретном изображении.

Выбор порога методом треугольника проиллюстрирован на рис. 2.12. Сначала в этом методе ищется самый высокий пик на гистограмме яркости. Далее проводится прямая из вершины пика в точку, которая соответствует черному цвету, и ищется самая удаленная от этой прямой точка гистограммы. Она будет лежать около основания этого пика. И в конце эта точка отодвигается на 20% в сторону черного цвета. Эта константа была подобрана автором метода экспериментально и неплохо работает для большого количества изображений.

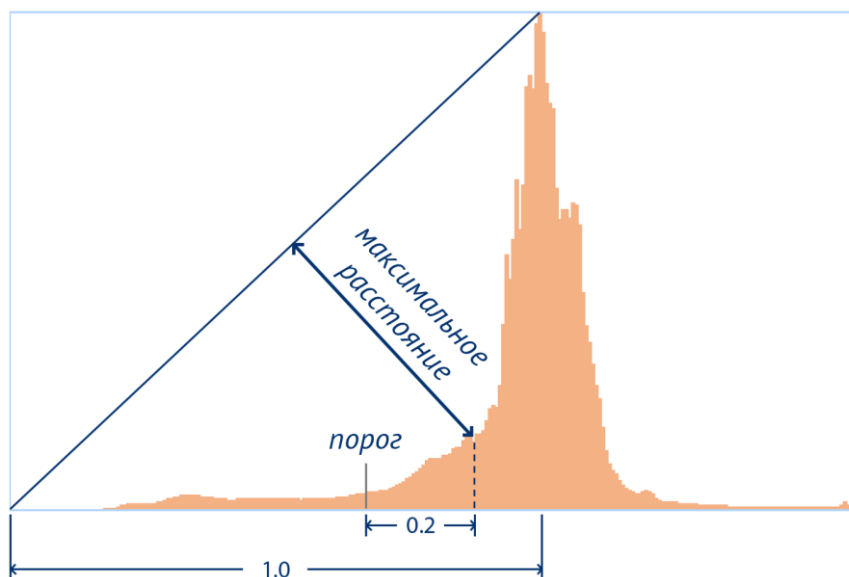


Рисунок 2.12 – Метод треугольника

Выбор порога методом Оцу схематично изображен на рис. 2.13.

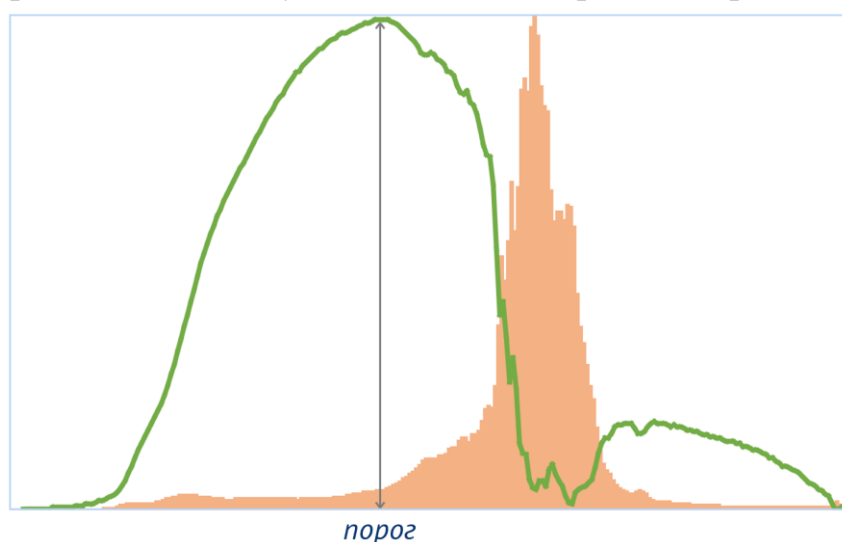


Рисунок 2.13 – Метод Оцу

Метод Оцу подбирает порог таким образом, чтобы две группы точек были максимально не похожи друг на друга. Для этого он вычисляет дисперсию между двумя группами точек: меньше и больше пороговой яркости. Вероятность конкретной яркости тем выше, чем больше точек с такой яркостью есть на изображении. Далее метод Оцу перебирает все возможные значения порога и для каждого вычисляет описанную выше дисперсию. В качестве порога выбирается значение с максимальной дисперсией, потому что дисперсия как раз отражает степень отличия двух групп точек друг от друга.

Адаптивное вычисление порога яркости пикселя в зависимости от яркости соседей в OpenCV можно использовать с помощью функции **adaptiveThreshold()**.

```
img_new = cv2.adaptiveThreshold(  
    img,                # входное изображение  
    maxVal,             # максимально возможное значение яркости  
    adaptiveMethod,     # метод расчета порога  
    type,               # тип преобразования  
    blockSize,          # размер окна  
    C,                  # константа  
)
```

В первый аргумент функции – **img** – передается изображение, к пикселям которого нужно применить порог. Так же, как и в случае с функцией **threshold()**, оно изменено не будет. Результат преобразования – возвращаемое значение функции. Во второй аргумент – **maxVal** – передается значение, которое будет присвоено пикселям в случае, если для адаптивно рассчитанного порога будет выбрано пороговое преобразование с бинаризацией. Аргумент **adaptiveMethod** задает способ расчета порога. На нем мы остановимся позже. Аргумент **type** принимает те же значения, что и аргумент **type** функции **threshold()**. Аргумент **blockSize** определяет, яркость каких соседей будет учитываться при адаптивном расчете. И последний аргумент – **C** – некоторая константа, которая будет вычитаться из значения порога, рассчитанного выбранным адаптивным методом. Она нужна для того, чтобы на практике немного увеличить или уменьшить порог на одинаковое значение для каждого пикселя, если результат расчетов по какой-то причине недостаточно хорошо подходит для решения задачи.

Теперь разберемся, как выполняется расчет значения яркости пикселя. Принцип расчета очень похож на то, как меняется значение пикселя при наложении фильтра. Для расчета в аргумент **adaptiveMethod** можно передать два значения:

- 1) `cv2.ADAPTIVE_THRESH_MEAN_C`;
- 2) `cv2.ADAPTIVE_THRESH_GAUSSIAN_C`.

В случае если выбран первый вариант, на пиксель накладывается фильтр размером **blockSize** на **blockSize** пикселей, каждое значение которого равно 1. Якорная точка фильтра будет находиться в центре. Из этого следует, что в аргумент **blockSize** нужно передавать нечетное число. Пороговое значение яркости

для пикселя, который совпадает с якорной точкой, будет рассчитываться как средневзвешенное по всем пикселям, на которые наложен фильтр. Если в аргумент **C** передано ненулевое число, то далее из полученного средневзвешенного будет вычтено значение **C**. Таким образом, порог яркости для данного пикселя получен. Он будет использован в качестве значения **T** в преобразовании, тип которого указан в аргументе **type**. Причем **T** рассчитывается отдельно для каждого пикселя изображения.

В случае если выбран второй аргумент, принцип расчета остается тем же. Меняются только значения в фильтре, который применяется к пикселю. Если в первом случае все значения были равны 1, то в этом случае имеет место распределение по Гауссу. Существует еще одно отличие, которое уже влияет на скорость работы. В случае с фильтром, заполненным одинаковыми значениями, нет смысла при сдвиге пересчитывать значения по всей площади: можно пересчитать только новые столбцы. В случае, если в фильтре значения разные, необходимо выполнять все вычисления заново при каждом сдвиге.

2.3 ФИЛЬТРЫ

Модуль 2.3 посвящен вопросам применения фильтров для сглаживания изображений. В первой части модуля рассматривается, в каких задачах нужно применять сглаживание, а также приводятся примеры простого линейного и простого нелинейного фильтра. Во второй части модуля рассматриваются более сложные фильтры, применяемые для сглаживания на практике, а также вопросы применения пользовательских фильтров.

2.3.1 Сглаживание изображений. Часть 1

Часто на практике изображение, которое мы получаем с помощью камеры, даже преобразованное в одноканальное, нельзя сразу применить для выполнения алгоритма обработки – вычисления границ, поиска фигур, даже пороговых преобразований. Это связано с тем, что в реальности на изображениях часто встречаются шумы. Шумы могут быть обусловлены как особенностями камеры, так и особенностями сцены.

Рассмотрим изображение, которое мы применяли ранее для работы с пороговыми преобразованиями: вид камеры робота на разметку пешеходного перехода – рис. 2.14.

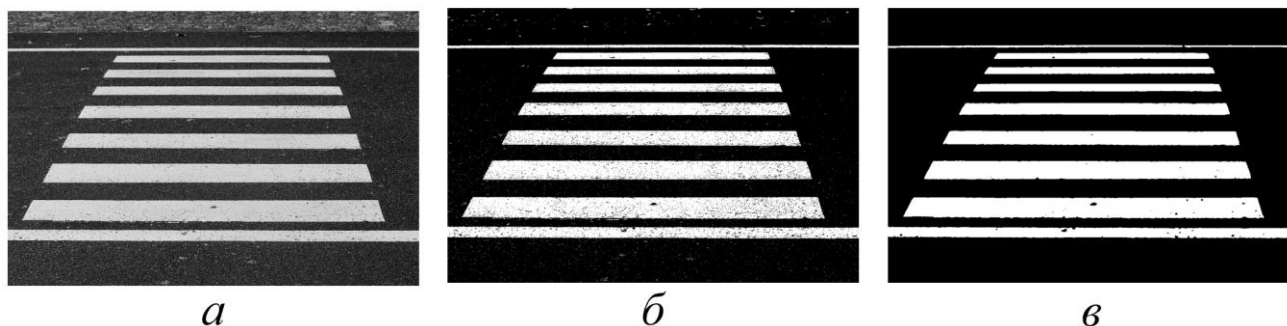


Рисунок 2.14 – Применение порогового преобразования:

а – исходное изображение; б – преобразование без сглаживания; в – преобразование со сглаживанием

Как можно заметить, на рис. 2.14, б белые точки отмечены не только на разметке, но и на дорожном полотне. При этом их достаточно много, чтобы увеличить вычислительную нагрузку и в некоторых ситуациях, возможно, привести к некорректной работе части алгоритмов. На рис. 2.14, в приведен результат применения порогового преобразования после сглаживания. Такое изображение лучше подходит для использования на следующих этапах, так как содержит меньше белых точек вне разметки и черных на разметке. Такого результата уда-

лось добиться с помощью применения к исходному изображению операции **сглаживания** перед выполнением непосредственно порогового преобразования.

Итак, **сглаживание**, или **размытие**, довольно часто встречается на этапе обработки изображений, поскольку позволяет удалить шумы и улучшить результат работы алгоритмов. Поэтому разберем, как можно реализовать такую операцию средствами OpenCV.

Самый простой вариант размытия реализован в OpenCV в виде функции **blur()**.

```
img_new = cv2.blur(  
    img,      # входное изображение  
    ksize,    # размер ядра  
    anchor,   # положение якорной точки  
    bType,    # тип рамки  
)
```

Такой фильтр вычисляет для каждого пикселя нормированное среднее значение по всем пикселям. В аргумент **img** передается входное изображение. Размер фильтра (ядра) задается во втором аргументе – **ksize**. В этот аргумент нужно передать кортеж из двух элементов – ширины и высоты фильтра.

Положение якорной точки можно указать, передав его в аргумент **anchor**. По умолчанию якорная точка находится в центре фильтра. Тип границы можно передать в аргумент **bType**, если имеется необходимость изменить тип по умолчанию – **cv2.BORDER_DEFAULT** – на какой-либо другой. Значение **cv2.BORDER_WRAP** не поддерживается. Возвращаемое значение функции – новое изображение. Функцию можно применять как к одноканальным, так и к многоканальным изображениям. Во втором случае каждый канал обрабатывается отдельно.

Данная функция является частным случаем **прямоугольного фильтра**, который в OpenCV реализован в функции **boxFilter()**.

```
img_new = cv2.boxFilter(  
    img,      # входное изображение  
    ddepth,   # глубина изображения-результата  
    ksize,    # размер ядра  
    anchor,   # положение якорной точки  
    normalize, # нормирование (если True)
```

bType # тип рамки

)

Прямоугольным называется такой фильтр, все элементы которого равны. Аргументы **img**, **ksize**, **anchor** и **bType** соответствуют одноименным аргументам функции **blur()**. Наличие аргумента **ddepth** – первое отличие функции **boxFilter()** от функции **blur()**. В функции **blur()** глубина выходного изображения совпадает с глубиной входного. Такое же поведение будет у функции **boxFilter()**, если передать в этот аргумент значение -1. При необходимости можно указать любое другое поддерживаемое значение глубины.

Данный фильтр может быть **нормированным** и **ненормированным**. Этот параметр регулируется аргументом **normalize**: нормированный, если значение аргумента – *True*, и ненормированный, если значение аргумента – *False*. По умолчанию аргумент имеет значение *True*. В первом случае значение каждого элемента будет делиться на площадь фильтра, то есть сумму его элементов. Пример ненормированного прямоугольного фильтра уже приводился, например, на рис. 2.1, а. Для прямоугольного фильтра размером 5 на 5, все элементы которого равны 1, нормирование будет осуществляться делением каждого элемента на 25.

На рис. 2.15 приведен пример применения прямоугольного нормированного фильтра размером 11 на 11 к трем вариантам исходного изображения.

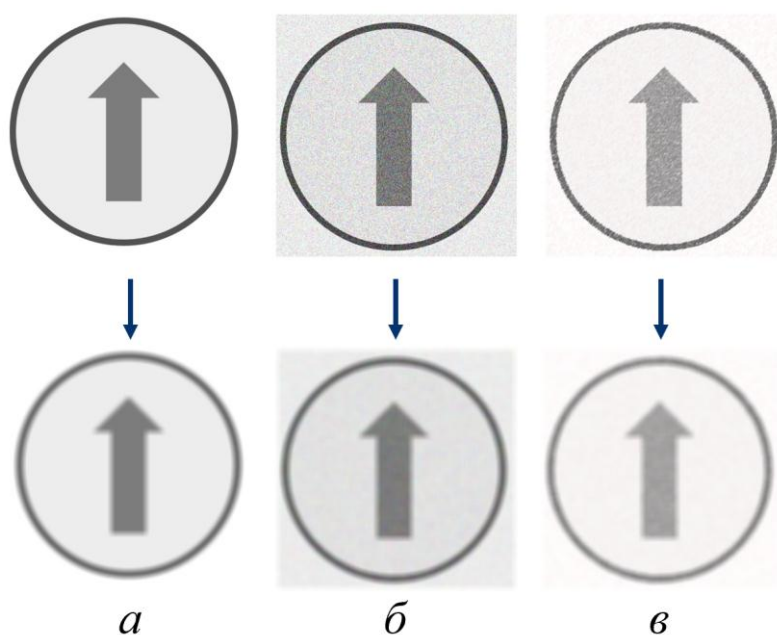


Рисунок 2.15 – Применение прямоугольного фильтра:

а – к исходному изображению; б – к изображению с шумом (вариант 1); в – к изображению с шумом (вариант 2)

Среди фильтров, позволяющих осуществить сглаживание изображения, существует также один пример **нелинейного** фильтра – **медианный фильтр**. При применении этого фильтра значения пикселя заменяются на медианное значение, а не на среднее арифметическое. Для каждого пикселя рассматриваются значения внутри прямоугольной области, которая определяется размером фильтра. Достоинством медианного фильтра перед прямоугольным является способность игнорировать выбросы – пиксели, значения которых сильно отличаются от соседних. В случае с расчетом среднего арифметического их значения могут оказать существенное влияние на результат. Так как в медианном фильтре выбирается точка со средним значением, такие выбросы на результат не влияют. В OpenCV медианный фильтр можно применить с помощью функции **medianBlur()**.

```
img_new = cv2.medianBlur(  
    img,      # входное изображение  
    ksize     # размер ядра  
)
```

Единственная пара параметров, которые можно указать для этой функции – входное изображение и размер ядра. Входное изображение задается так же, как для других фильтров – передается в аргумент **img**. А вот задание размера ядра отличается. В аргумент **ksize** здесь нужно передать не кортеж из двух элементов, а целое число. Возвращаемое значение функции – новое изображение. Результат применения данного фильтра к тому же набору изображений представлен на рис. 2.16.

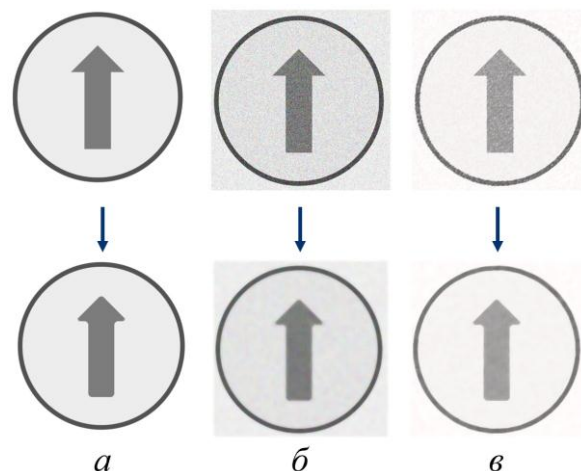


Рисунок 2.16 – Применение медианного фильтра:

а – к исходному изображению; б – к изображению с шумом (вариант 1); в – к изображению с шумом (вариант 2)

По сравнению с прямоугольным фильтром медианный фильтр тоже успешно справился с удалением шумов, но не привел к потере четкости границ.

2.3.2 Сглаживание изображений. Часть 2

На практике для сглаживания часто применяются фильтры на основе нормального распределения. Одним из наиболее популярных способов сглаживания изображения является фильтр Гаусса. Для каждого пикселя вычисляется взвешенное среднее значение самого пикселя и его соседей. Веса в функции зависят от расстояния между центральным пикселем и соответствующим соседним.

В OpenCV он реализован в функции **GaussianBlur()**. При применении данной функции будет использовано нормированное ядро.

```
img_new = cv2.GaussianBlur(  
    img,      # входное изображение  
    ksize,    # размер ядра  
    sigmaX,   # сигма по оси X  
    sigmaY,   # сигма по оси Y  
    bType     # тип рамки  
)
```

Аргументы **img**, **ksize** и **bType** совпадают с одноименными аргументами для функций **blur()** и **boxFilter()**.

Величина сигмы влияет на то, какие соседи имеют больший вес. Чем больше сигма, тем больше вес удаленных точек и тем меньше вес центральных. Ядро в некотором смысле «растягивается», более равномерно распределяя веса по всем соседям. Если же значение сигмы маленькое, то точки около центра ядра будут иметь очень большое значение, а удаленные от центра точки будут иметь вес, практически равный нулю. Можно отдельно задавать сигму для расстояний по оси *X* и по оси *Y*. Примеры ядер для разных параметров сигма можно увидеть на рис. 2.17. Для выбора одинаковых значений параметров сигма не обязательно задавать значения в оба аргумента – **sigmaX** и **sigmaY**. В случае если **sigmaY** равно 0, этот параметр выбирается равным **sigmaX**.

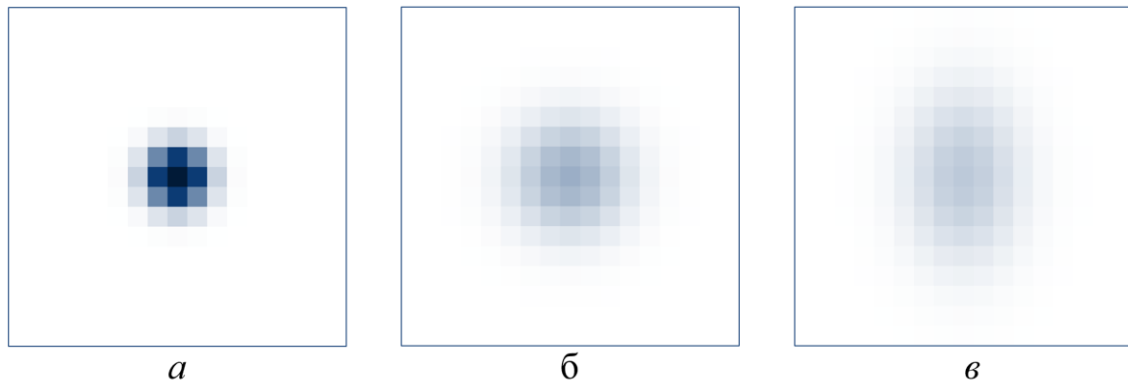


Рисунок 2.17 – Графические представления фильтра 17 на 17 для значений:
а – $\sigma_X = 1, \sigma_Y = 1$; б – $\sigma_X = 2, \sigma_Y = 2$; в – $\sigma_X = 2, \sigma_Y = 3$

В случае если в оба аргумента передать значения 0, расчет значений будет выполняться по формуле 2.5:

$$\begin{aligned} \sigma_x &= \left(\frac{n_x - 1}{2} \right) \cdot 0,30 + 0,80, \quad n_x = ksize_w - 1 \\ \sigma_y &= \left(\frac{n_y - 1}{2} \right) \cdot 0,30 + 0,80, \quad n_y = ksize_h - 1 \end{aligned} \quad (2.5)$$

где σ_x – значение для аргумента **sigmaX**; σ_y – значение для аргумента **sigmaY**, $ksize_w$ – ширина фильтра; $ksize_h$ – высота фильтра. Таким образом, параметры вычисляются по размеру окна. Графическое представление фильтра приведено на рис. 2.18.



Рисунок 2.18 – Графическое представление фильтра 17 на 17 с параметрами нормального распределения, зависящими от размера фильтра

Результат применения фильтра Гаусса размером 11 на 11 представлен на рис. 2.19.

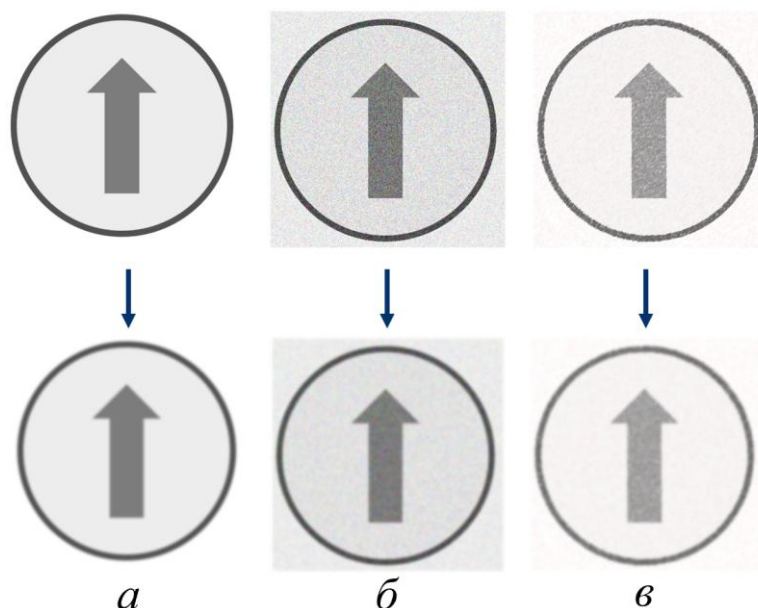


Рисунок 2.19 – Применение фильтра Гаусса:

а – к исходному изображению; б – к изображению с шумом (вариант 1); в – к изображению с шумом (вариант 2)

В данном примере значение аргументов **sigmaX** и **sigmaY** – 0. Для ядер размеров 3 на 3, 5 на 5 и 7 на 7 со значением аргумента **sigmaX** – 0 вычисление фильтра Гаусса в OpenCV оптимизировано.

Еще один способ сглаживания, доступный в OpenCV – двусторонняя фильтрация. Этот способ схож со сглаживанием по Гауссу, но также имеет важное отличие. Сглаживание по Гауссу подразумевает, что на реальных изображениях яркость меняется постепенно, вследствие чего существует зависимость между яркостями соседних пикселей. Тогда шум, яркость которого отличается резко, уменьшается вследствие сглаживания. Однако такой способ работает хуже, когда пересчитывает яркость пикселей, которые находятся на границе областей с разным уровнем яркости. Поэтому в результате применения сглаживания по Гауссу границы на изображении размываются. Такое поведение может быть нежелательным. Двусторонний фильтр позволяет выполнить сглаживание изображения с сохранением границ.

Двусторонний фильтр тоже считает для каждого пикселя средневзвешенное значение в зависимости от яркости пикселя и его соседей. Важное отличие двустороннего фильтра от фильтра Гаусса заключается в его весовой функции.

Первый компонент весовой функции двустороннего фильтра учитывает расстояние между центральным пикселем и текущим соседним. Второй компонент основан на разнице между яркостью центрального пикселя и данного со-

седнего. Такой фильтр может быть полезен при подготовке изображения к сегментации. В OpenCV двусторонний фильтр реализован в функции **bilateralFilter()**.

```
img_new = cv2.bilateralFilter(  
    img,          # входное изображение  
    d,            # размер окрестности пикселя  
    sigmaColor,   # ширина второго компонента весовой функции  
    sigmaSpace,   # ширина первого компонента весовой функции  
    bType        # тип рамки  
)
```

Параметры **img** и **bType** принимают те же значения, что и в других реализациях фильтров сглаживания. В аргумент **d** передается число – диаметр окрестности пикселя, для которого выполняется расчет значения. По сути, это то же, что размер ядра **ksize**, только в случае двустороннего фильтра ширина будет равна высоте. В аргумент **sigmaColor** передается значение ширины компонента весовой функции, который учитывает яркость пикселей. И в компонент **sigmaSpace** передается значение ширины компонента, отвечающего за учет расстояния. Этот параметр так же влияет на весовую функцию, как параметры **sigmaX** и **sigmaY** функции **CaussianBlur()** – конечно, если они одинаковые. Возвращаемое значение функции – новое изображение. Пример работы функции приведен на рис. 2.20.

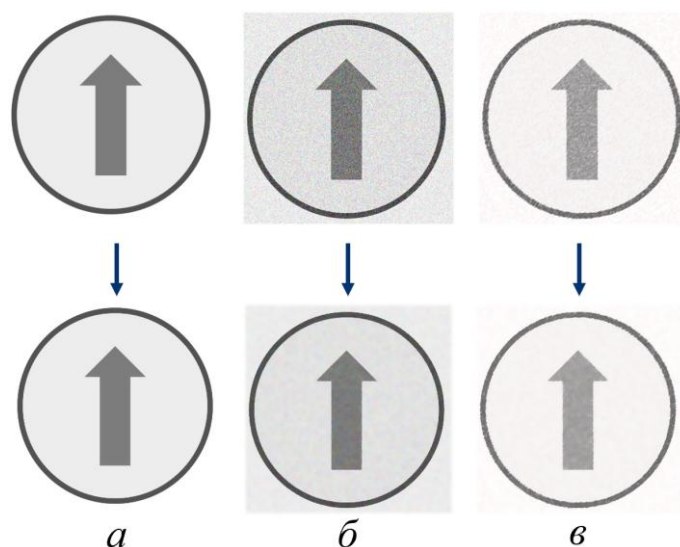


Рисунок 2.20 – Применение двустороннего фильтра:

а – к исходному изображению; б – к изображению с шумом (вариант 1); в – к изображению с шумом (вариант 2)

Действительно, двусторонний фильтр сохраняет четкость границ. Однако он не предназначен для применения в тех задачах, где сглаживание, наоборот, необходимо именно на границах. Такие случаи характерны для задач поиска границ и вычисления контуров. Существенным недостатком двустороннего фильтра является низкая скорость работы, так как вычислительная нагрузка достаточно большая. Поэтому он редко применяется в обработке видеопотока.

Стоит также отметить, что перечень фильтров, реализованных в OpenCV, не ограничивается упомянутыми выше вариантами. Например, существует ряд фильтров, связанных с вычислением производных. Они будут подробнее рассмотрены в следующих модулях.

Также помимо свертки с указанными фильтрами OpenCV предоставляет возможность использовать свертку с произвольным линейным фильтром. Чтобы создать фильтр, нужно создать двумерный массив нужного размера, каждый элемент которого будет числом с плавающей точкой. Для этой задачи можно воспользоваться инструментами библиотеки NumPy: например, **ones()** или **full()**. Чтобы фильтр был нормированным, необходимо будет вручную разделить значения на сумму элементов фильтра. Применить фильтр можно с помощью функции **filter2D()**.

```
img_new = cv2.filter2D(  
    img,      # входное изображение  
    ddepth,   # глубина изображения  
    kernel,   # сгенерированное ядро  
    anchor,   # положение якоря  
    delta,    # добавляемое значение  
    bType     # тип границы  
)
```

Параметры **img**, **ddepth**, **anchor** и **bType** принимают те же значения, что и в случае с другими фильтрами. В аргумент **kernel** передается переменная, которая хранит ядро. Параметр **delta** задавать не обязательно. В случае если для данного параметра указано значение, оно будет добавлено к каждому пикселю результирующего изображения после применения фильтра. Возвращаемое значение функции – новое изображение.